



CHINESISCHER RESTSATZ, KRYPTOGRAPHIE

Chinesischer Restsatz. Für welche Zahlen $x \in \mathbb{Z}$ gilt

1. $x \equiv 3 \pmod{2}$,
 $x \equiv 3 \pmod{5}$,
 $x \equiv 3 \pmod{7}$.
2. $x \equiv 1 \pmod{2}$,
 $x \equiv 3 \pmod{4}$.
3. Was sind die jeweils kleinsten positiven x in 1. und 2.?

Lösung.

Eigener Lösungsversuch.

Rechnen mit großen Zahlen mit CRS. In C kann man mittels des Datentyps `unsigned long long` (nur) 64-bit Zahlen darstellen, also Zahlen x mit

$$0 \leq x \leq 2^{64} - 1 = 18.446.744.073.709.551.615.$$

In der Kryptographie braucht man aber viel größere Zahlen (z.B. in RSA hat der Modul mindestens 1024 Bit, also 309 Dezimalstellen!). Eine Möglichkeit zur Darstellung und dem Rechnen mit großen Zahlen bietet der Chinesische Restsatz:

Wenn $m_1, m_2, \dots, m_n$ paarweise teilerfremd (und groß, aber noch im Datentyp darstellbar!) sind und $m = m_1 m_2 \cdots m_n$ (riesengroß!), so kann jede Zahl x mit $0 \leq x \leq m$ eindeutig durch ihre Reste $a_k = x \bmod m_k$ ($1 \leq k \leq n$) repräsentiert werden:

$$x \leftrightarrow (a_1, \dots, a_n).$$

Beispiel: $m_1 = 9$, $m_2 = 8$. Dann ist etwa $39 = (3, 7)$, denn $39 = 3 \pmod{9}$ und $39 = 7 \pmod{8}$. Umgekehrt kann zu jedem Tupel mithilfe des chinesischen Restsatzes die Zahl rekonstruiert werden: man erhält $x = 39$ als eindeutige Lösung von

$$x \equiv 3 \pmod{9},$$

$$x \equiv 7 \pmod{8}.$$

Wir addieren und multiplizieren jetzt x, y nicht direkt, sondern rechnen mit den Resten: Sind $x \leftrightarrow (a_1, \dots, a_n)$, $y \leftrightarrow (b_1, \dots, b_n)$ zwei Zahlen, so entspricht

$$x + y \leftrightarrow ((a_1 + b_1) \bmod m_1, \dots, (a_n + b_n) \bmod m_n),$$

$$x \cdot y \leftrightarrow ((a_1 \cdot b_1) \bmod m_1, \dots, (a_n \cdot b_n) \bmod m_n).$$

Fazit: Anstatt mit den großen Zahlen x, y zu rechnen, wird mit den kleineren Resten gerechnet und am Ende das Ergebnis mit dem chinesischen Restsatz zurückgerechnet. (Die einzelnen Reste können parallel berechnet werden, Stichworte: Nebenläufigkeit und Multicore-Rechner! In der Praxis verwendet man für die Module m_k Zahlen der Form $2^l - 1$, da sich die Modulo-Rechnung für diese Zahlen binär gut implementieren lässt.)

Jetzt zur eigentlichen **Aufgabe** - wir rechnen leider nur exemplarisch mit kleinen Zahlen :-). Angenommen, Ihre Lieblingsprogrammiersprache kann nur 4-bit Zahlen x mit

$$0 \leq x \leq 2^4 - 1 = 15$$

darstellen. Sie möchten aber auch 9-bit Zahlen x mit

$$0 \leq x \leq 2^9 - 1 = 511$$

addieren und multiplizieren.

1. Wählen Sie passende Module, um nach obigem Verfahren
2. $459 + 510$ und
3. $459 \cdot 510$ zu berechnen.

Lösung.

Eigener Lösungsversuch.

Lineare Kryptographie. Es wird mit $E(x) = (a \cdot x + b) \bmod 26$ verschlüsselt. Dabei ist $0 \leq x < 26$ die Nummer des zu verschlüsselnden Buchstabens (0-25 entspricht a-z).

1. Sei $a = 3, b = 2$. Verschlüsseln Sie „informatik“ und entschlüsseln Sie „ismvkhob“.
2. Geben Sie die Entschlüsselung als Funktion $D(x)$ an. Welche Zahlen $0 \leq a, b < 26$ sind für eine eindeutige Ver-/Entschlüsselung geeignet?

Lösung.

Eigener Lösungsversuch.